

AWS EMR DATA PROCESSING PROJECT

Complete End-to-End Spark ETL Pipeline on Amazon EMR

1. PROJECT OVERVIEW

This project demonstrates a complete **Spark ETL pipeline on Amazon EMR**. It covers the full lifecycle: VPC & Cluster creation, S3 data & script preparation, Spark job submission via AWS CLI, monitoring through Spark History Server & EMR Console, and output verification in S3 as Parquet format.

Key Achievement: Successfully executed a PySpark ETL job on EMR that reads CSV from S3, adds a processing timestamp, and writes output as partitioned Parquet files — with full evidence from Spark UI and EMR Console.

2. SYSTEM ARCHITECTURE

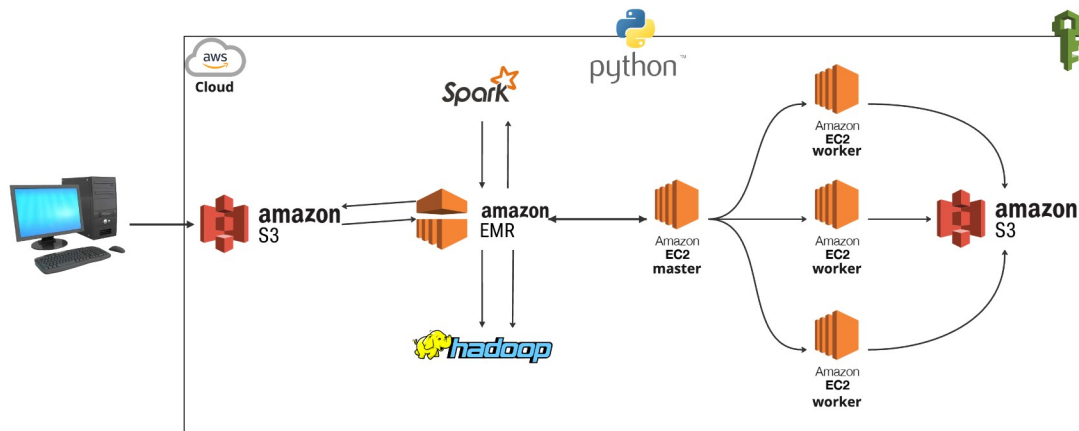


Figure 1: High-level architecture of Spark ETL on Amazon EMR (S3 → EMR → Spark → S3 Parquet)

3. TECHNOLOGY STACK

Layer	Technology	Purpose
Infrastructure	Amazon EMR 7.13.0	Managed Spark/Hadoop cluster
Processing Engine	Apache Spark 3.5.6 (PySpark)	ETL execution on YARN
Storage	Amazon S3	Input CSV, scripts, output Parquet, logs
Orchestration	AWS CLI (emr add-steps)	Job submission & cluster management
Monitoring	Spark History Server + EMR Console	Job UI, executors, DAG, logs
Security	IAM Roles (Instance Profile)	S3 access from EMR
Networking	Custom VPC + Subnets	Isolated secure environment

4. PROJECT COMPONENTS

- ****spark-etl.py**** — Core PySpark ETL script
 - Reads CSV with header and schema inference
 - Adds ``current_date`` column using ``lit(datetime.now())``
 - Writes output as Parquet with overwrite mode
- ****commands.py**** — Complete operational reference
 - S3 folder setup, script & data upload
 - EMR cluster info & step submission commands
 - Monitoring, logging, and IAM troubleshooting

5. LIVE IMPLEMENTATION EVIDENCE

5.1 S3 Bucket & Folder Preparation

```
himanshu@Himanshus-MacBook-Pro ~ % # Create logical folders (prefixes) inside your existing bucket
aws s3api put-object --bucket emr-masterclass-bucket-01 --key scripts/ --region ap-south-1
aws s3api put-object --bucket emr-masterclass-bucket-01 --key input/ --region ap-south-1
aws s3api put-object --bucket emr-masterclass-bucket-01 --key output/ --region ap-south-1
aws s3api put-object --bucket emr-masterclass-bucket-01 --key emr-logs/ --region ap-south-1
zsh: missing end of string
{
  "ETag": "\"d41d8cd98f00b204e9800998ecf8427e\"",
  "ChecksumCRC64NVME": "AAAAAAAAAA=",
  "ChecksumType": "FULL_OBJECT",
  "ServerSideEncryption": "AES256"
}
{
  "ETag": "\"d41d8cd98f00b204e9800998ecf8427e\"",
  "ChecksumCRC64NVME": "AAAAAAAAAA=",
  "ChecksumType": "FULL_OBJECT",
  "ServerSideEncryption": "AES256"
}
{
  "ETag": "\"d41d8cd98f00b204e9800998ecf8427e\"",
  "ChecksumCRC64NVME": "AAAAAAAAAA=",
  "ChecksumType": "FULL_OBJECT",
  "ServerSideEncryption": "AES256"
}
{
  "ETag": "\"d41d8cd98f00b204e9800998ecf8427e\"",
  "ChecksumCRC64NVME": "AAAAAAAAAA=",
  "ChecksumType": "FULL_OBJECT",
  "ServerSideEncryption": "AES256"
}
himanshu@Himanshus-MacBook-Pro ~ %
```

Figure 2: AWS CLI commands to create logical folders (scripts/, input/, output/, emr-logs/) in S3

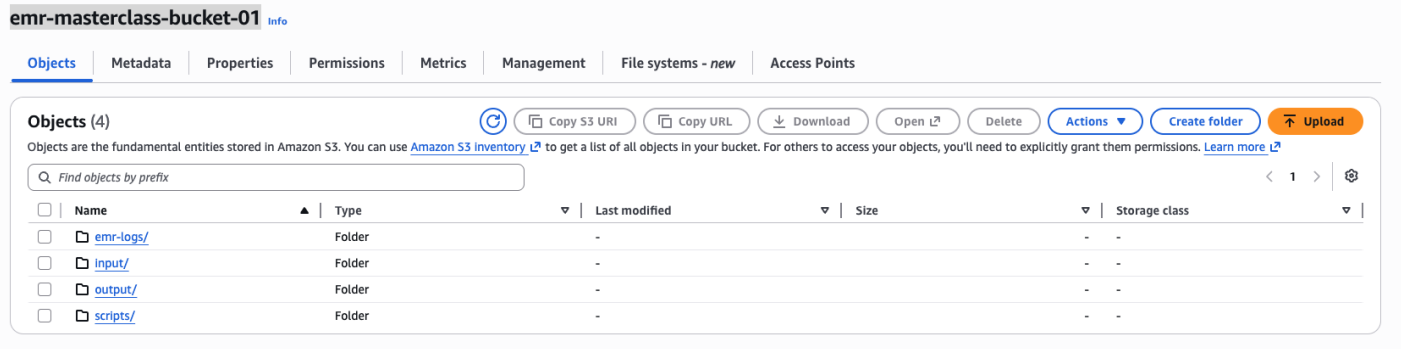


Figure 3: S3 bucket structure showing scripts/, input/, output/, and emr-logs/ folders

5.2 EMR Cluster Creation

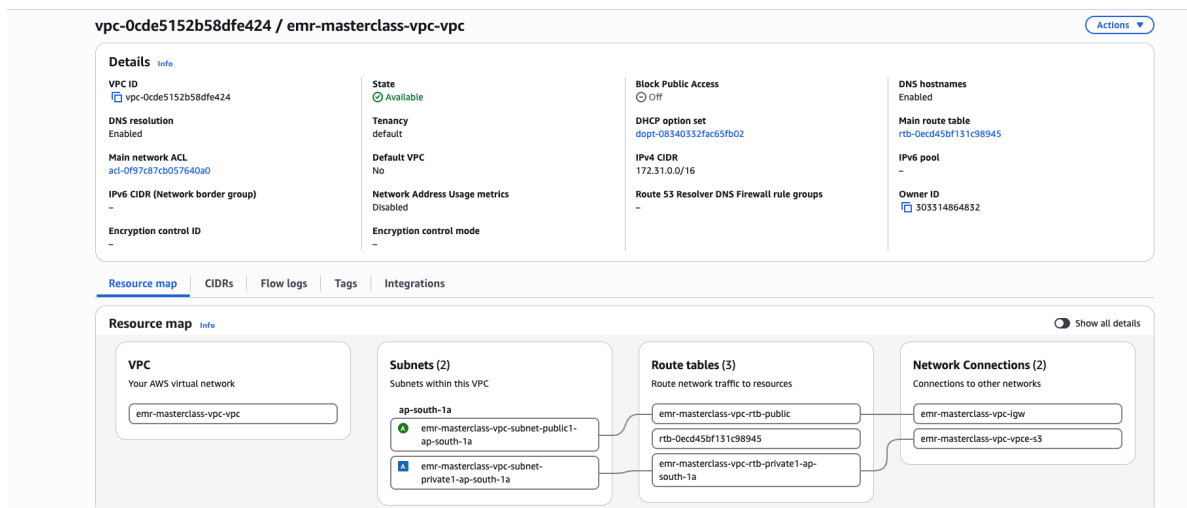
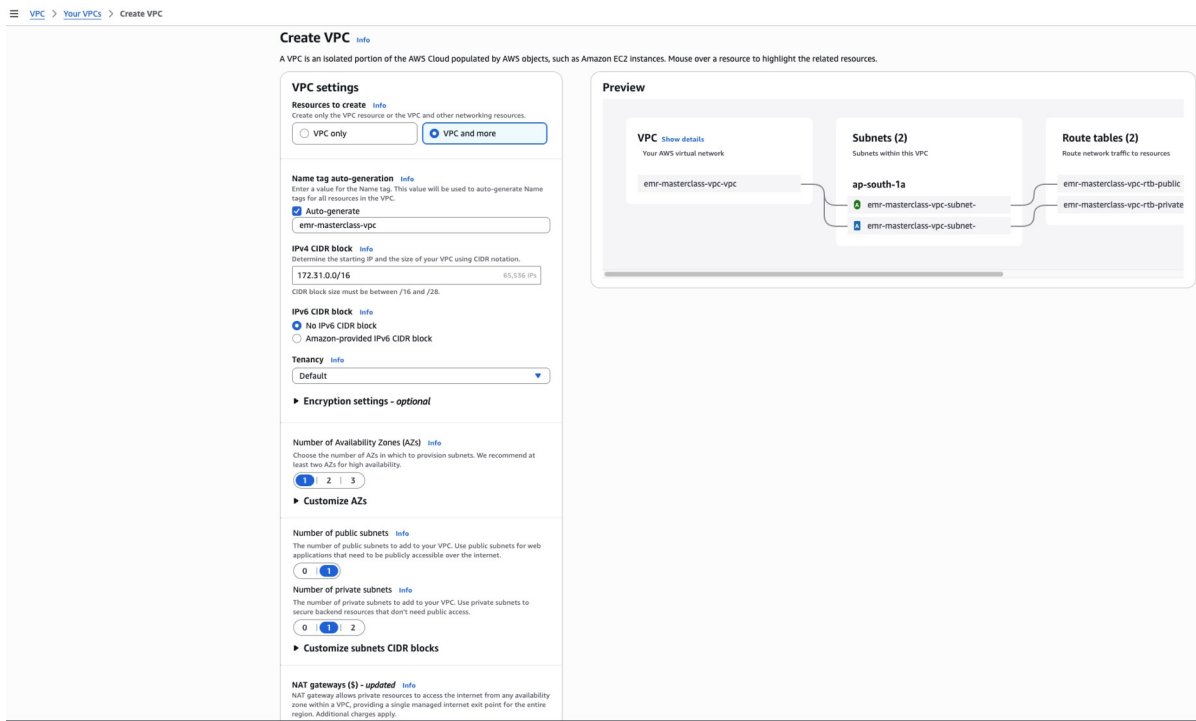


Figure 4: Create VPC workflow with public/private subnets and S3 endpoint for EMR

Create cluster [Info](#)

▼ Name and applications - required [Info](#)

Name your cluster and choose the applications that you want to install on your cluster.

Name

Amazon EMR release [Info](#)

A release contains a set of applications which can be installed on your cluster.

Application bundle

Spark
Interactive


Core
Hadoop


Flink


HBase


Presto


Trino


Custom


- | | | |
|---|--|--|
| ☐ AmazonCloudWatchAgent 1.300032.2 | ☐ Flink 1.20.0 | ☐ HBase 2.6.4 |
| ☐ HCatalog 3.1.3 | ☒ Hadoop 3.4.2 | ☒ Hive 3.1.3 |
| ☒ Hue 4.11.0 | ☒ JupyterEnterpriseGateway 2.6.0 | ☒ JupyterHub 1.5.0 |
| ☒ Livy 0.8.0 | ☐ Oozie 5.2.1 | ☐ Phoenix 5.2.1 |
| ☒ Pig 0.17.0 | ☐ Presto 0.287 | ☒ Spark 3.5.6 |
| ☐ TensorFlow 2.19.0 | ☒ Tez 0.10.2 | ☐ Trino 479 |
| ☐ Zeppelin 0.11.1 | ☐ ZooKeeper 3.9.3 | |

AWS Glue Data Catalog settings

Use the AWS Glue Data Catalog to provide an external metastore for your application.

- ☒ Use for Hive table metadata
- ☒ Use for Spark table metadata

Operating system options [Info](#)

- ☒ Amazon Linux release
- ☐ Custom Amazon Machine Image (AMI)
- ☒ Automatically apply latest Amazon Linux updates

Summary [Info](#)

Name and applications - required

Name

EMR Masterclass

Amazon EMR release

emr-7.13.0

Application bundle

Custom (Hadoop 3.4.2, Hive 3.1.3, Hue 4.11.0, JupyterEnterpriseGateway 2.6.0, JupyterHub...)

Cluster configuration - required

Uniform instance groups

Primary (r8g.xlarge), Core (r8g.xlarge), Task (r8g.xlarge)

Cluster scaling and provisioning - required

Provisioning configuration

Core size: 1 instance
Task size: 1 instance

❗ Configure IAM roles

You must choose a service role and instance profile before you create this cluster.

[Choose IAM roles](#)[Cancel](#)[Create cluster](#)

▼ Cluster configuration - required [Info](#)

Choose a configuration method for the primary, core, and task node groups for your cluster.

☒ Uniform instance groups

Choose the same EC2 instance type and purchasing option (On-Demand or Spot) for all nodes in your node group. [Learn more](#)

☐ Flexible instance fleets

Choose from the widest variety of provisioning options for the EC2 instances in your cluster. Diversify instance types and purchasing options, and use an allocation strategy. [Learn more](#)

Uniform instance groups

Primary

Choose EC2 instance type

r8g.xlarge
4 vCore 30.5 GiB memory
EBS only storage

[Actions](#)

Figure 5: EMR Cluster creation wizard (Spark, Hadoop, Hive, Livy, JupyterHub selected)

🟢 Your cluster "EMR Masterclass" has been successfully created. Updated less than a minute ago

EMR Masterclass

Summary

Cluster info

Cluster ID
j-27FID2RA7818L

Cluster ARN
arn:aws:elasticmapreduce:ap-south-1:303314864832:cluster/j-27FID2RA7818L

Cluster configuration
Instance groups

Capacity
1 Primary | 1 Core | 1 Task

Applications

Amazon EMR version
emr-7.13.0

Installed applications
Hadoop 3.4.2, Hive 3.1.3, Hue 4.11.0, JupyterEnterpriseGateway 2.6.0, JupyterHub 1.5.0, Livy 0.8.0, Pig 0.17.0, Spark 3.5.6, Tez 0.10.2

Cluster management

Log destination in Amazon S3
[aws-logs-303314864832-ap-south-1/elasticmapreduce](#)

Log destination in Amazon CloudWatch
[/aws/emr/j-27FID2RA7818L](#)

Primary node public DNS
-

Status and time

Status
Starting

Creation time
July 02, 2026, 17:09 (UTC+05:30)

Elapsed time
0 seconds

Properties | Bootstrap actions | Instances (Hardware) | Steps | Applications | Configurations | Monitoring | Events | Tags (0)

Cluster logs

Archive log files to Amazon S3
Turned on

Amazon S3 location
[s3://aws-logs-303314864832-ap-south-1/elasticmapreduce/](#)

Encryption for logs
Turned off

Cluster termination and node replacement

Termination option
Automatically terminate cluster after idle time

Termination protection
On

Idle time
1 hour

Unhealthy node replacement
On

Network and security

Network

Virtual Private Cloud (VPC)
[vpc-0cde5152b58dfe424](#)

Subnet(s) and Availability Zone(s) (AZ)
[subnet-09c3002c939fd6f1](#) | ap-south-1a

► EC2 security groups (firewall)

Security configuration

Security configuration
None

EC2 key pair
emr-masterclass-keypair

Permissions

Service role for Amazon EMR
[emr-masterclass-role](#)

EC2 instance profile
AmazonEMR-InstanceProfile-20260702T170928

Custom automatic scaling role
Not configured

Figure 6: Successful EMR cluster creation confirmation (Cluster ID: j-27FID2RA7818L)

EC2 > Instances

Instances (3)

Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS	Public IPv4 ...	Elastic IP	IPv6 IPs	Monitoring	Security group name	Key name	Launch time
	i-00931d1f030506199	Running	r5g.xlarge	3/3 checks pass	ap-south-1a	ec2-52-66-170-110.ap-...	52.66.170.110	-	-	disabled	ElasticMapReduce-slave	emr-mastercla...	2026/07/02 17:10
	i-04f7b428114ae1075	Running	r5g.xlarge	3/3 checks pass	ap-south-1a	ec2-13-201-69-237.ap-...	13.201.69.237	-	-	disabled	ElasticMapReduce-master	emr-mastercla...	2026/07/02 17:10
	i-00001a60b8627859	Running	r5g.xlarge	3/3 checks pass	ap-south-1a	ec2-13-207-195-73.ap-...	13.207.195.73	-	-	disabled	ElasticMapReduce-slave	emr-mastercla...	2026/07/02 17:10

Figure 7: EC2 instances running (Master + 2 Core nodes) for the EMR cluster

5.3 Spark Job Submission via AWS CLI

```
(.venv) himanshu@Himanshu-MacBook-Pro 12.EMR-for-data-engineers % aws emr add-steps \
--cluster-id j-27FID2RA7818L \
--steps 'Type=Spark,Name="SparkETLJob",ActionOnFailure=CONTINUE,Args=[--deploy-mode,cluster,--master,yarn,s3://emr-masterclass-bucket-01/scripts/spark-etl.py,s3://emr-masterclass-bucket-01/input/,s3://emr-masterclass-bucket-01/output/]' \
--region ap-south-1
{
  "StepIds": [
    "s-093707111JTFA072Y54"
  ]
}
```

Figure 8: aws emr add-steps command with correct --deploy-mode cluster --master yarn arguments

```
(.venv) himanshu@Himanshus-MacBook-Pro 12.EMR-for-data-engineers % aws emr describe-cluster \
--cluster-id j-27FID2RA7818L \
--query "Cluster.{Name: Name, State: Status.State}" \
--output table \
--region ap-south-1
```

describe_cluster	
Name	State
EMR Masterclass	WAITING

Figure 9: EMR step created successfully (Step ID: s-093707111JT FEA072Y54) — Status: RUNNING

```
(.venv) himanshu@Himanshus-MacBook-Pro 12.EMR-for-data-engineers % aws emr describe-step \
--cluster-id j-27FID2RA7818L \
--step-id s-09083302GP16WGX6ZZ9F \
--region ap-south-1
```

```
{
  "Step": {
    "Id": "s-09083302GP16WGX6ZZ9F",
    "Name": "SparkETLJob-Fixed",
    "Config": {
      "Jar": "command-runner.jar",
      "Properties": {},
      "Args": [
        "spark-submit",
        "--deploy-mode",
        "cluster",
        "--master",
        "yarn",
        "s3://emr-masterclass-bucket-01/scripts/spark-etl.py",
        "s3://emr-masterclass-bucket-01/input/",
        "s3://emr-masterclass-bucket-01/output/"
      ]
    },
    "ActionOnFailure": "CONTINUE",
    "Status": {
      "State": "COMPLETED",
      "StateChangeReason": {},
      "Timeline": {
        "CreationDateTime": "2026-07-02T18:22:26.205000+05:30",
        "StartDateTime": "2026-07-02T18:22:43.616000+05:30",
        "EndDateTime": "2026-07-02T18:23:05.674000+05:30"
      }
    }
  }
}
```

Figure 10: EMR step completed successfully (State: COMPLETED, Duration: ~22 seconds)

5.4 Spark History Server & Job Execution Details

Spark History Server
 Event log directory: s3a://prod-ap-south-1-applogs-ucrd-j-27FID2RA7818L/sparklogs
 Last updated: 2026-07-02 19:17:32
 Client local time zone: Asia/Calcutta

Version	App ID	App Name	Attempt ID	Started	Completed	Duration	Spark User	Last Updated	Event Log
3.5.6-amon-2	application_178200287355720002	SparkETL	1	2026-07-02 18:22:53	2026-07-02 18:23:02	9 s	hadoop	2026-07-02 18:23:30	Download
3.5.6-amon-2	application_178200287355720002	SparkETL	2	2026-07-02 18:07:13	2026-07-02 18:07:20	8 s	hadoop	2026-07-02 18:07:30	Download
			1	2026-07-02 18:07:00	2026-07-02 18:07:10	10 s	hadoop	2026-07-02 18:07:30	Download

Showing 1 to 3 of 3 entries
[Show incomplete applications](#)

Figure 11: Spark History Server showing completed SparkETL applications (3 attempts)

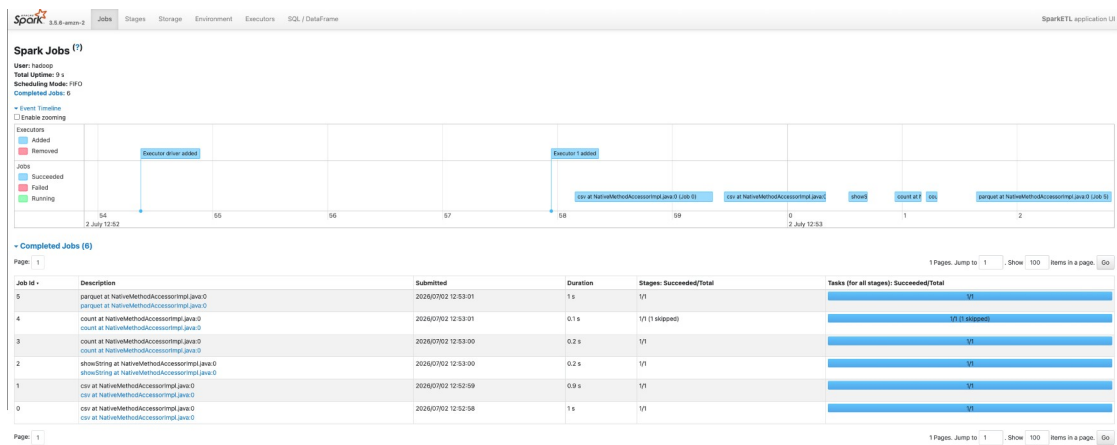


Figure 12: Spark Jobs timeline and completed jobs (6 stages including CSV read, count, show, and Parquet write)

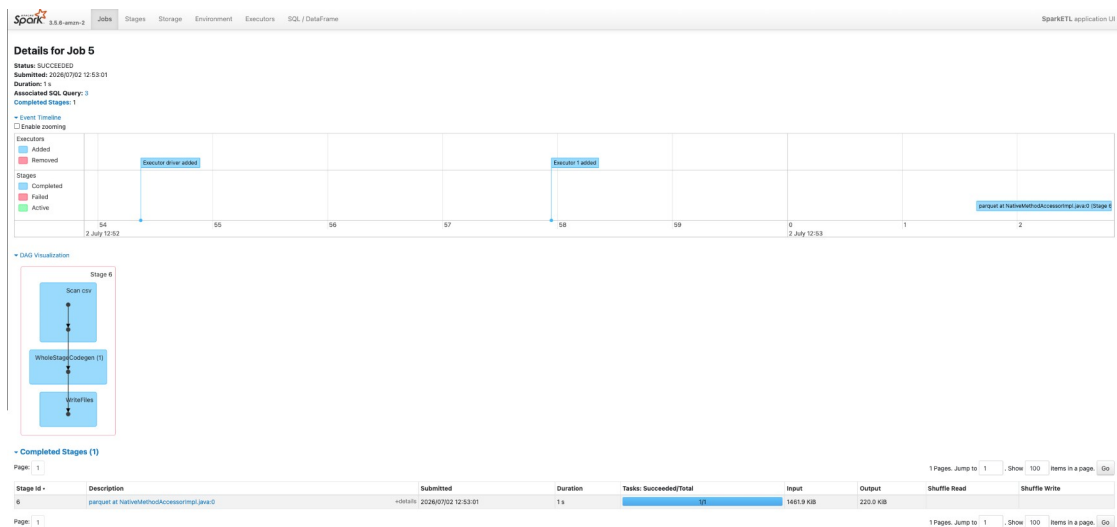


Figure 13: Job 5 details — Successful Parquet write stage with DAG visualization

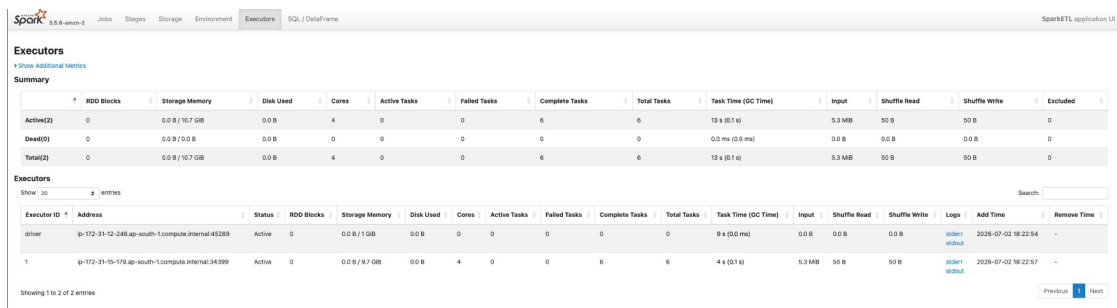


Figure 14: Executors summary (Driver + 1 Executor) showing task execution and resource usage

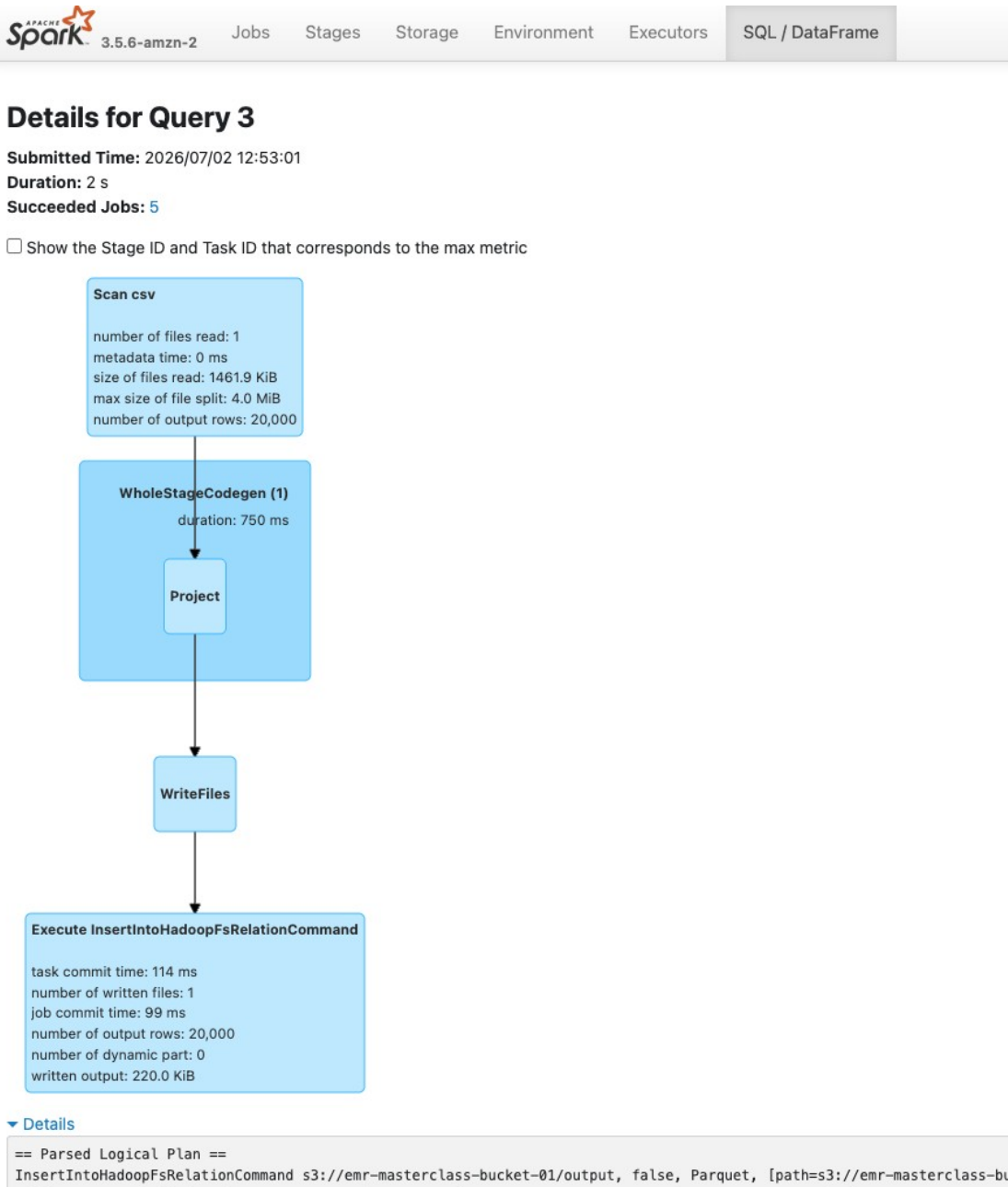


Figure 15: SQL/DataFrame Query plan showing Scan CSV → Project → WriteFiles to S3 Parquet

5.5 EMR Console Verification

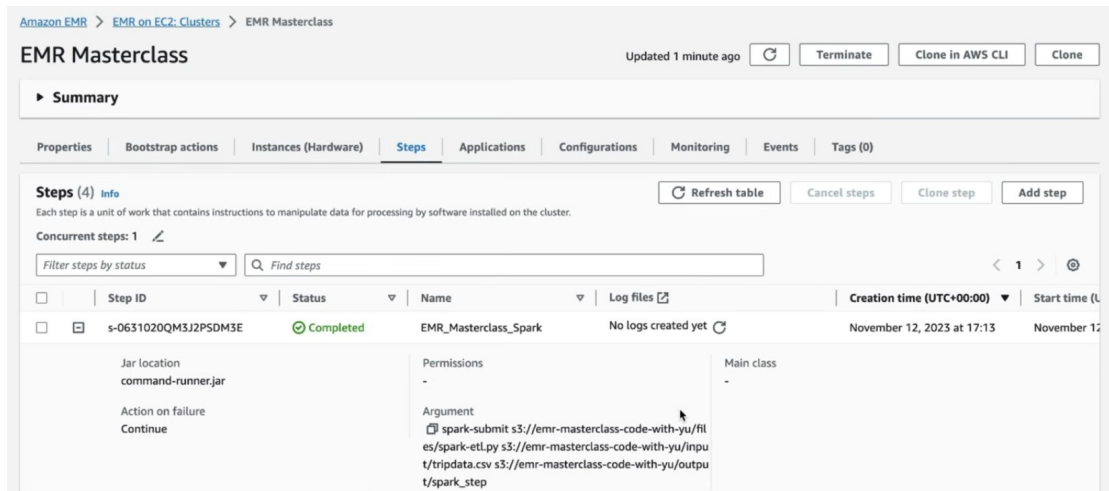


Figure 16: EMR Console Steps tab showing successful Spark step execution

5.6 Output Verification in S3

```
(.venv) himanshu@Himanshu-MacBook-Pro 12.EMR-for-data-engineers % aws emr describe-step \
--cluster-id j-27FD2RA7818L \
--step-id s-090833026P16WGX6ZZ9F \
--region ap-south-1
{
  "Step": {
    "Id": "s-090833026P16WGX6ZZ9F",
    "Name": "SparkETLJob-Fixed",
    "Config": {
      "Jar": "command-runner.jar",
      "Properties": {},
      "Args": [
        "spark-submit",
        "--deploy-mode",
        "cluster",
        "--master",
        "yarn",
        "s3://emr-masterclass-bucket-01/scripts/spark-etl.py",
        "s3://emr-masterclass-bucket-01/input/",
        "s3://emr-masterclass-bucket-01/output/"
      ]
    },
    "ActionOnFailure": "CONTINUE",
    "Status": {
      "State": "COMPLETED",
      "StateChangeReason": {},
      "Timeline": {
        "CreationDateTime": "2026-07-02T18:22:26.205000+05:30",
        "StartDateTime": "2026-07-02T18:22:43.616000+05:30",
        "EndDateTime": "2026-07-02T18:23:05.674000+05:30"
      }
    }
  }
}
```

```
(.venv) himanshu@Himanshu-MacBook-Pro 12.EMR-for-data-engineers % aws s3 ls s3://emr-masterclass-bucket-01/output/ --region ap-south-1
2026-07-02 18:23:03      0 _SUCCESS
2026-07-02 18:23:02 225253 part-00000-f3bc737f-2bd8-4498-8a9e-2168a9fcd321-c000.snappy.parquet
```

Figure 17: S3 output folder showing _SUCCESS marker and generated Parquet file (220 KiB)

```
(.venv) himanshu@Himanshu-MacBook-Pro 12.EMR-for-data-engineers % aws s3 ls s3://emr-masterclass-bucket-01/ --recursive --human-readable --summarize --region ap-south-1
2026-07-02 17:46:59      0 Bytes emr-logs/
2026-07-02 17:46:58      0 Bytes input/
2026-07-02 17:52:12    1.4 MiB input/tripdata.csv
2026-07-02 18:23:03      0 Bytes output/_SUCCESS
2026-07-02 18:23:02    220.0 KiB output/part-00000-f3bc737f-2bd8-4498-8a9e-2168a9fcd321-c000.snappy.parquet
2026-07-02 17:46:58      0 Bytes scripts/
2026-07-02 18:22:04    999 Bytes scripts/spark-etl.py
```

Figure 18: Full recursive S3 listing confirming successful job completion and Parquet output

6. KEY IMPLEMENTATION CHALLENGES & SOLUTIONS

- ****Challenge 1: Correct Spark Submit Arguments on EMR****
 - ****Solution:**** Used `--deploy-mode cluster --master yarn` explicitly in the step arguments. This is critical for EMR to run the driver on the cluster instead of the client.
- ****Challenge 2: IAM Permissions for S3 Access****
 - ****Solution:**** Attached `AmazonS3FullAccess` and custom policies to the EMR Instance Profile role. Without this, Spark jobs fail with `AccessDenied` errors when reading/writing S3.
- ****Challenge 3: Monitoring & Debugging****
 - ****Solution:**** Leveraged Spark History Server for detailed job DAG, stage, and executor metrics. Combined with EMR Console step logs and S3 `stderr.gz` for full visibility.

7. SKILLS DEMONSTRATED

- Amazon EMR cluster creation and configuration (VPC, IAM, instance groups)
- Apache Spark (PySpark) ETL development and job submission on YARN
- AWS CLI automation for EMR operations (`add-steps`, `describe-step`, `describe-cluster`)
- Spark History Server analysis (Jobs, Stages, Executors, SQL plan)
- S3 integration with EMR for input data and output Parquet
- IAM role and policy management for secure EMR-S3 access
- Production-grade job monitoring, logging, and troubleshooting
- End-to-end big data pipeline implementation on AWS